

Module-2

Fundamentals of C programming:

Branching and Looping- IF statement, IF..else statement, Nesting of if..Else statement, Else if ladder, Switch Statement, The? : Operator, Go To statement, While Do while, For, break and continue statements.

Arrays - One, Two & multidimensional arrays.

Strings – Introduction to strings.

Advanced features-Type casting, type conversion.

Formatted and unformatted Input/Output- scanf(), printf(), gets(), puts(), getch() and putch().

KRITI KAUSIK

Branching and Looping:

C provides two styles of flow control:

- Branching
 - Looping
- **Branching is deciding what actions to take, and looping is deciding how many times to take a certain action.**

Branching: Branching is so called because the program chooses to follow one branch or another. It allows you to execute different blocks of code depending on whether a given condition is true or false.

Looping: Loops provide a way to repeat commands and control how many times they are repeated. It's used to automate repetitive tasks, iterate through collections of data, and perform actions a specific number of times. Different types of loops provide flexibility in how you control the repetition.

Control statements:

- Control statements in C help the computer execute a certain logical statement and decide whether to enable the control of the flow through a certain set of statements or not.
- Also, it is used to direct the execution of statements under certain conditions.

☐ **Decision-making control statements**

1. Simple if statement
2. If-else statements
3. Nested if-else statements
4. else-if ladder

☐ **Conditional Control Statements**

☐ **Goto statements in C**

☐ **Loop control statements in C**

1. While Loop
2. Do-while Loop
3. For Loop

❑ Decision-making control statements:

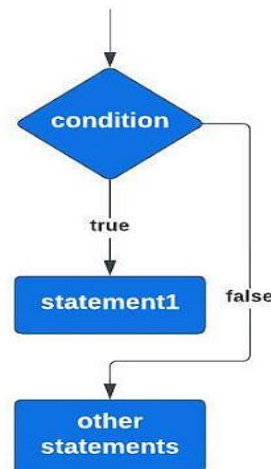
1. If statement- If statements are carried out to perform some operation when the condition is only true.

If the condition of the if statement is true, then the statements under the if block is executed else the control is transferred to the statements outside the if block.

Syntax:

```
if (condition)
{
    Statement1;
}
```

Flowchart:



```
1  #include <stdio.h>
2  void main()
3  {
4      int a,b;
5      printf("enter the numbers for a and b");
6      scanf("%d%d",&a,&b);
7      if(a>b)
8      {
9          printf("%d is greater than %d ",a,b);
10     }
11 }
12
```

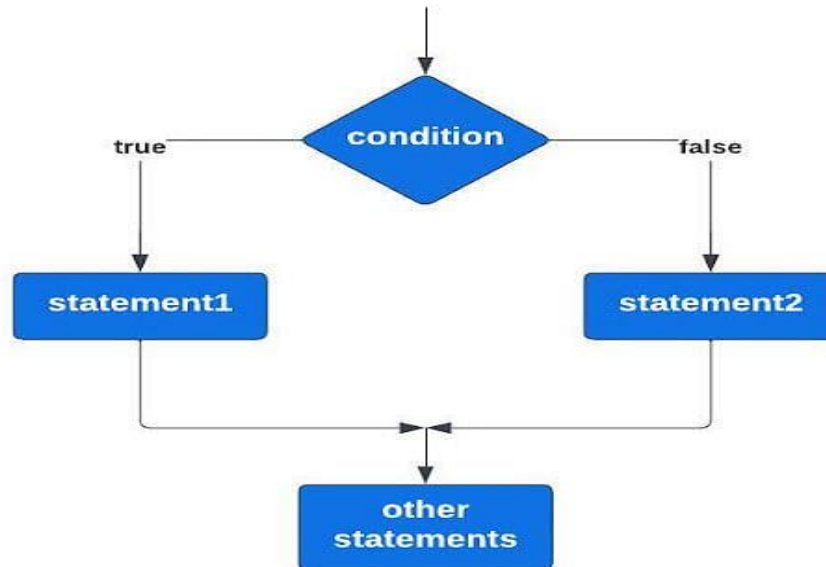
2.If-else Statement- In some situations, you may have to execute statements based on true or false under certain conditions, therefore; we use if-else statements.

If the condition is true, then if block will be executed otherwise the else block is executed.

Syntax:

```
if (condition)
    Statement1;
else
    Statement2;
```

Flow Chart:



```
1  #include <stdio.h>
2  void main()
3  {
4      int a,b;
5      printf("enter the numbers for a and b");
6      scanf("%d%d",&a,&b);
7      if(a>b)
8      {
9          .....
10         printf("%d is greater than %d ",a,b);
11     }
12     else
13     {
14         printf("%d is less than %d",a,b);
15     }
16
17 }
```

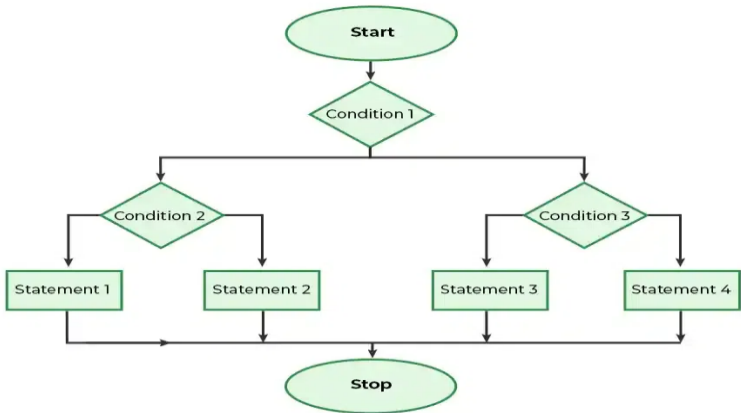
3.Nested if-else Statements- A nested if in C is an if statement that is the target of another if statement.

Nested if statements mean an if statement inside another if statement. Yes, both C and C++ allow us to nested if statements within if statements, i.e, we can place an if statement inside another if statement.

Syntax:

```
if (condition1)
{
    // Executes when condition1 is true
    if (condition2)
    {
        // Executes when condition2 is true
    }
    else
    {
        // Executes when condition2 is false
    }
}
```

Flowchart:



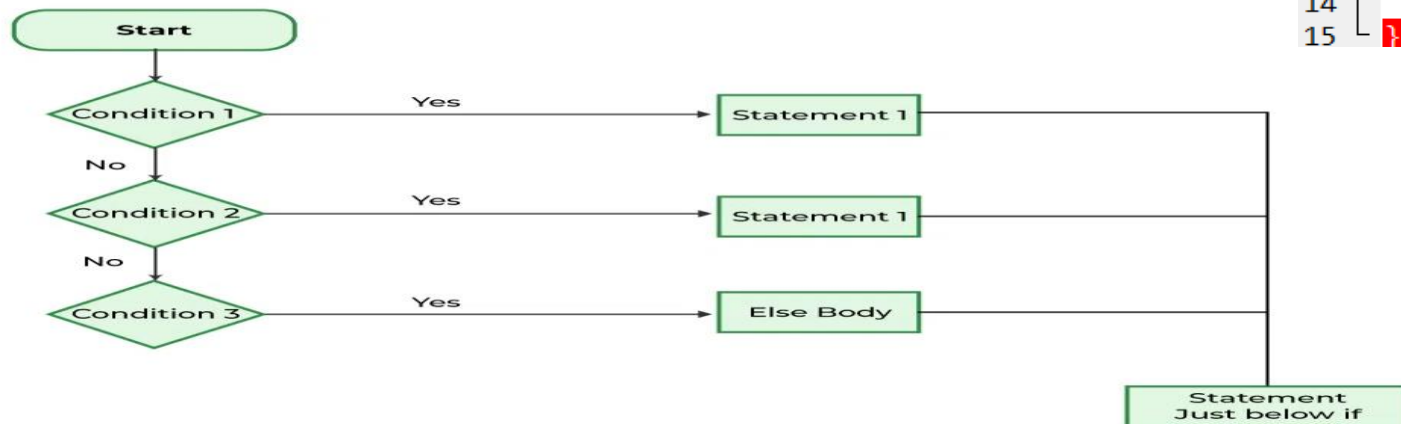
```
1  #include <stdio.h>
2  void main()
3  {
4      int a,b;
5      printf("enter the numbers for a and b");
6      scanf("%d%d",&a,&b);
7      if(a!=b)
8      {
9          printf("%d is not equal to %d ",a,b);
10
11          if(a>b)
12          {
13              printf("%d is greater than %d",a,b);
14          }
15          else
16          {
17              printf("%d is less than %d",a,b);
18          }
19      }
20      else
21      {
22          printf("%d is equal to %d ",a,b);
23      }
24  }
25  }
```

4. Else-if Ladder Statements- The else-if ladder statements contain multiple else-if, when either of the condition is true the statements under that particular “if” will be executed otherwise the statements under the else block will be executed.

If none of the conditions is true, then the final else statement will be executed. if-else-if ladder is similar to the switch statement.

Syntax:

```
if (condition)
    statement;
else if (condition)
    statement;
.
.
else
    statement;
```



```
Untitled1.c  x
1  #include <stdio.h>
2
3  int main()
4  {
5      int i = 20;
6
7      if (i == 10)
8          printf("i is 10");
9      else if (i == 15)
10         printf("i is 15");
11     else if (i == 20)
12         printf("i is 20");
13     else
14         printf("i is not present");
15 }
```

❑ Conditional Statements: (Switch)

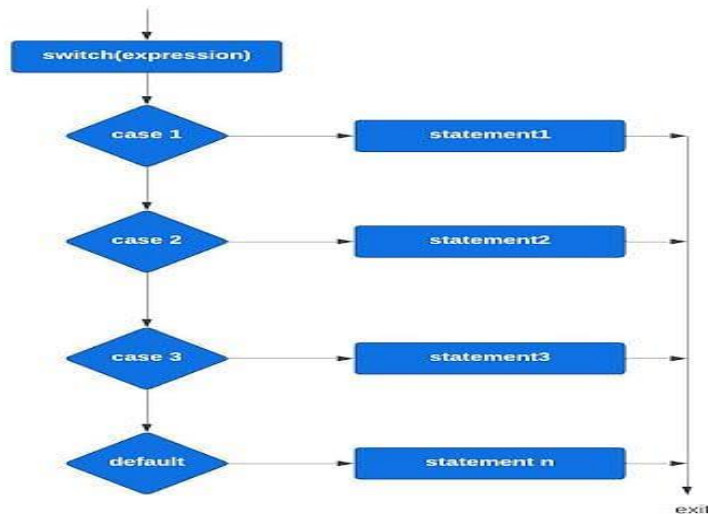
- The switch case statement is an alternative to the if else if ladder that can be used to execute the conditional code based on the value of the variable specified in the switch statement.

The switch block consists of cases to be executed based on the value of the switch variable. As per the value of the switch expression, the switch statement will allow multi-way branching.

Syntax:

```
Switch (expression)
{
  Case label 1:  statement1;
                -----
                break;
  Case label 2:  statement2;
                -----
                break;
  Case label 3:  statement3;
                -----
                break;
  -----
  Case label n:  statement n;
                -----
                break;
  default:      default statement;
}
Other statements;
```

Flow Chart:



demo3.c

```
1  #include <stdio.h>
2  void main()
3  {
4      int n = 3;
5      printf("n=%d \n",n);
6      switch(n)
7      {
8          case 1: printf("the value of n is 1");
9              break;
10         case 2: printf("the value of n is 2");
11             break;
12         case 3: printf("the value of n is 3");
13             break;
14         default: printf("value is invalid");
15     }
16 }
17
```


Conditional Operator:

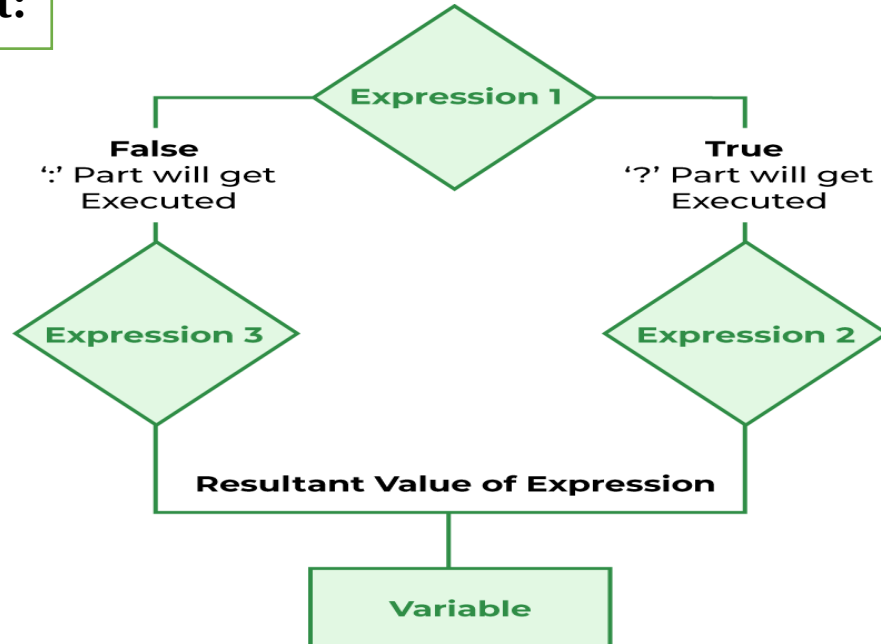
- The conditional operator is used to add conditional code in our program. It is similar to the if-else statement. It is also known as the ternary operator as it works on three operands.

Syntax:

(condition) ? [true_statements] : [false_statements];

if condition is true ? then X return value :
otherwise Y value;

Flowchart:



Goto Statements in C:

- The goto statements are used to transfer the flow of control in a program, goto statement is also known as a jump control statement because it is used to jump to the specified part of the program.

It is used to alter the normal sequence of program execution by transferring control to some other part of the program.

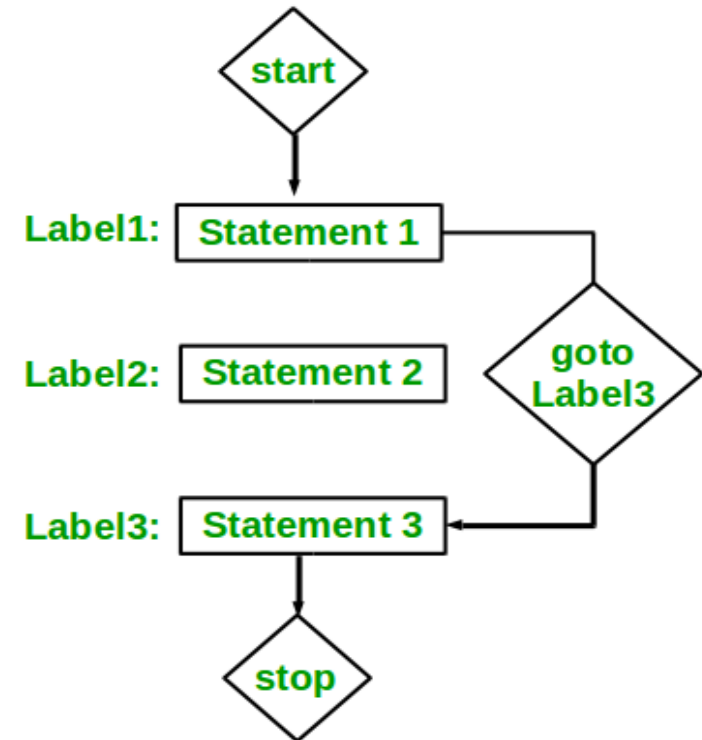
- The goto statement allows us to transfer control of the program to the specified label.

Syntax:

```
Syntax: goto labelname;
```

```
Example: goto odd;
```

Flowchart:



Loop Control Statements in C

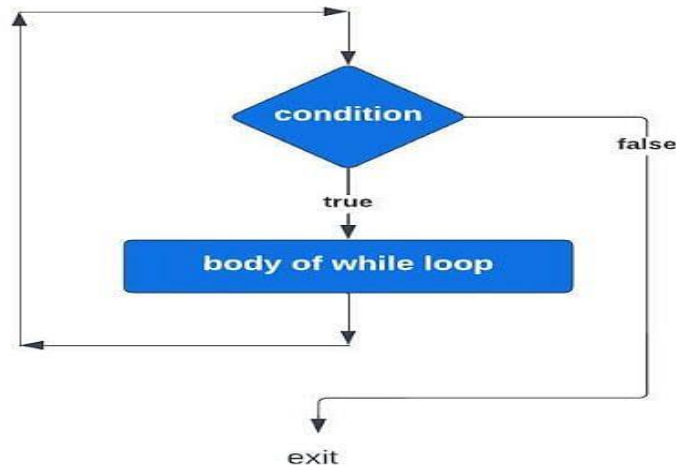
1.While Loop- A while loop is also known as an entry loop because in a while loop the condition is tested first then the statements underbody of the while loop will be executed.

If the while loop condition is false for the first time itself then the statements under the while loop will not be executed even once.

Syntax:

```
While (condition)
{
    Statement 1;
    Statement 2;
    -----
    -----
    Statement n;
}
```

Flowchart:



```
1  #include <stdio.h>
2  void main()
3  {
4      int a;
5      printf("enter the number for a");
6      scanf("%d",&a);
7      while(a<=5)
8      {
9          printf("%d \n",a);
10         ++a;
11     }
12 }
13
```

2. Do while loop:

- The do-while is also known as an exit loop because in the do-while loop, the statements will be executed first and then the condition is checked.

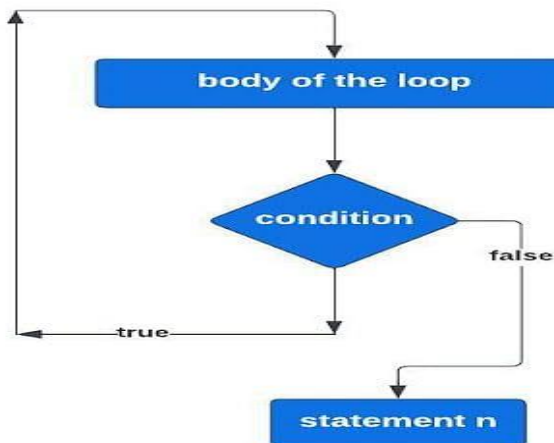
If the condition of the while loop is true, then the body of the loop will be executed again and again until the condition is false. Once the condition is false, the control will transfer outside the do-while loop and execute statements followed soon after the do-while loop.

Syntax:

```
do
{
    Statement 1;
    Statement 2;
    -----
    -----
}
while (condition);

Statement n;
```

Flowchart:



```
1  #include <stdio.h>
2  int main()
3  {
4      int a = 0;
5
6      do
7      {
8          printf("%d \n",a);
9          a++;
10     }
11     while(a<=5);
12     return 0;
13 }
```

3. For Loop:

The for loop is also known as a pre-test loop. From the following syntax, expression1 is an initialization, expression2 is the conditional expression and expression3 is an updation.

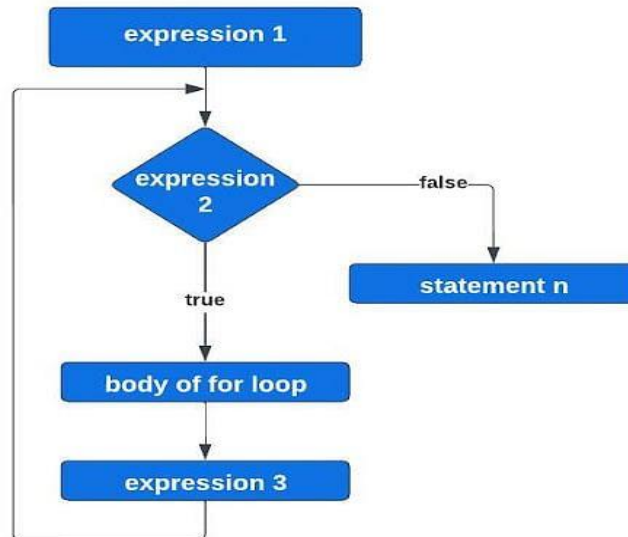
The variables can be initialized in for the statement itself.

Syntax:

```
for (expression1; expression2; expression3)
{
Statement1;
Statement2;
-----
}
Statement n ;
```

In the for loop, expression1 is used to initialize the variable, expression2 is evaluated and if the condition is true, then the body of for loop will be executed and then the statements under expression3 will be executed.

Flowchart:



```
1  #include <stdio.h>
2  void main()
3  {
4      int n = 4,i;
5      for( i=0; i<n; i++)
6      {
7          printf("%d \n",i);
8      }
9  }
```

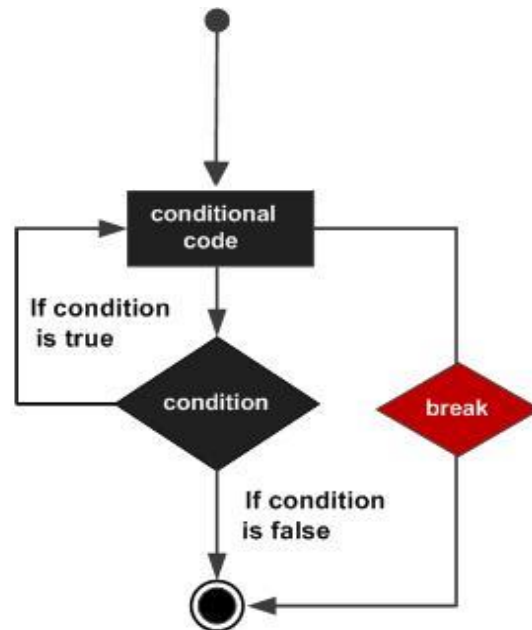
Break Statement:

This loop control statement is used to terminate the loop. As soon as the break statement is encountered from within a loop, the loop iterations stop there, and control returns from the loop immediately to the first statement after the loop.

Syntax:

`break;`

Flowchart:



```
for ( expression )  
{  
    statement1;  
    ....  
    if (condition) true  
        break;  
    ....  
    statement2;  
}
```

out of the loop

```
while (test condition)  
{  
    statement1;  
    ....  
    if (condition) true  
        break;  
    ....  
    statement2;  
}
```

out of the loop

Continue Statement:

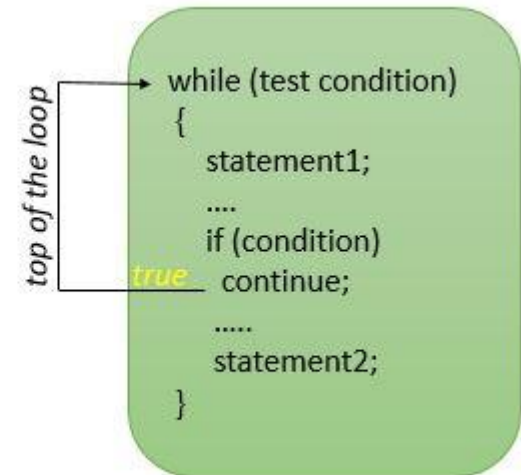
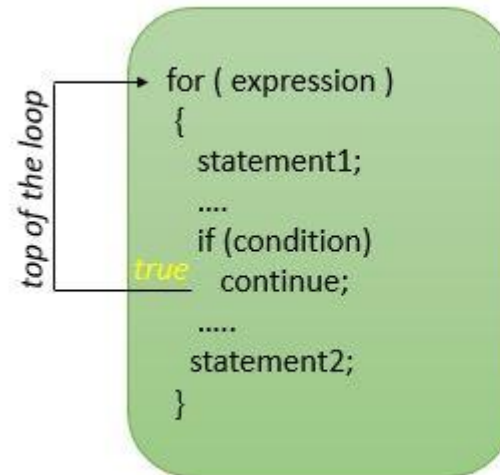
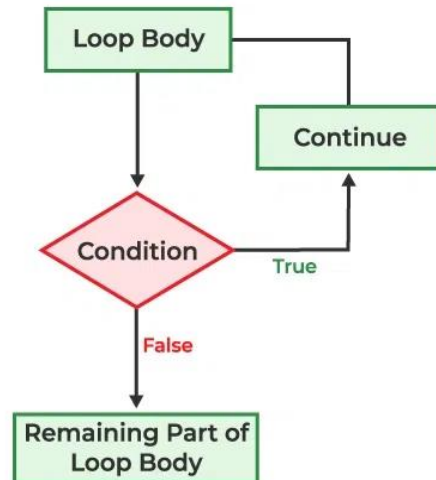
This loop control statement is just like the break statement. The continue statement is opposite to that of the break statement, instead of terminating the loop, it forces to execute the next iteration of the loop.

When the continue statement is executed in the loop, the code inside the loop following the continue statement will be skipped and the next iteration of the loop will begin.

Syntax:

continue;

Flowchart:



Arrays:

- Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.
- To create an array, define the data type (like int) and specify the name of the array followed by square brackets [].
- To insert values to it, use a comma-separated list, inside curly braces:

```
int myNumbers[] = {25, 50, 75, 100};
```

Access the Elements of an Array

To access an array element, refer to its index number.

Array indexes start with 0: [0] is the first element. [1] is the second element, etc.

This statement accesses the value of the first element [0] in myNumbers:

Ex:

```
int myNumbers[] = {25, 50, 75, 100};  
printf("%d", myNumbers[0]);
```


Change an Array Element:

```
myNumbers[0] = 33;  
  
int myNumbers[] = {25, 50, 75, 100};  
myNumbers[0] = 33;  
  
printf("%d", myNumbers[0]);
```

Loop Through an Array:

```
int myNumbers[] = {25, 50, 75, 100};  
int i;  
  
for (i = 0; i < 4; i++) {  
    printf("%d\n", myNumbers[i]);  
}
```

Set Array Size:

```
int myNumbers[4];  
  
myNumbers[0] = 25;  
myNumbers[1] = 50;  
myNumbers[2] = 75;  
myNumbers[3] = 100;
```

Types of Array in C:

There are mainly three types of arrays in C language :

- One – dimensional arrays (1D Array)
- Two – dimensional arrays (2D Array)
- Multi – dimensional arrays

1 D Array:- The normal array we make in C language is a one-dimensional array.

Syntax:-`data_type array_name[array_size];`

- Datatype- Indicates what kind of data is going to be stored in the array.
- array_name- Here we write the name of the array so that we can access and use the array with this name anywhere in our program.
- array_size- Here we write the size of array, suppose if I want to make array of 20 variables then I will write 20 here and if I want to make array of 200 variables then here, I will write 200.

Ex:`int number[100];`

int is the data type, number is the name of the array, and 100 is the size of that array.

Note :

- When the array is declared, the garbage value is stored in it by default.
- The index number of the array by default starts from zero, like if the size of the array is 5 then its index number will be something like arr[0], arr[1], arr[2], arr[3], arr[4].

Initialization of one -Dimensional Array In C:

1. Compile time initialization
2. Runtime initialization

1. Compile-time initialization of a One-Dimensional Array :

Just as we initialize a normal variable in C language, we can also initialize 1D Array in the same way and when we do this, it is called Compile-time Initialization.

When this program is compiled, then at the same time the value of this array will be initialized.

```
int var[10] = {10,20,30,40,50,60,70,80,90,100};
```

```
int var2[] = {101,201,301,401,501,601,701,801,901,};
```

```
int var3[10] = {11,22,33,44,55};
```

2. Runtime initialization of a One-Dimensional Array:

When in Array, the value is initialized at program run time then it is called run time initialization.

```
#include<stdio.h>
void main()
{
    int arr[4];
    int i, j;
    printf("Enter array element : ");
    for(i = 0; i < 4; i++)
    {
        scanf("%d", &arr[i]);    //Run time array initialization
    }
    printf("Printing array element : ");
    for(j = 0; j < 4; j++)
    {
        printf("%d\n", arr[j]);
    }
}
```

- **Accessing Elements of One-Dimensional Array In C:**

We can access the array element by array name and index number and can modify them accordingly.

```
#include<stdio.h>
void main()
{   int i;
    int arr[5] = {10,20,30,40,50};
    printf("Before any changes result :");
    for(i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }

    arr[3] = 45;
    printf("\nAfter changes result : ");
    for(i = 0; i < 5; i++)
    {
        printf("%d ",arr[i]);
    }
}
```

2. Two-Dimensional Array in C (2D Array)

Two Dimensions Array is like a Matrix. Which is represented in the form of rows and columns. In simple words, it is also called Array of Array.

Syntax : data_type array_name[rows][columns];

Example : int arr[21][30];

- **Initialization of Two-Dimensional Array In C(2D Array)**

1. Compile time initialization
2. Runtime initialization

1. Compile-time Initialization :

In compile-time initialization, we initialize two-dimensional array like this :

```
int matrix_A[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}};
```

2. Runtime Initialization of a Two Dimensional :

When the value of Two-Dimensional Array is initialized at run time then it is called Runtime initialization.

```
#include<stdio.h>
void main()
{
    int arr[4][4];
    int i, j, k;
    printf("Enter array element");
    for(i = 0; i <= 3; i++)
    {
        for(j = 0; j < 4; j++)
        {
            scanf("%d", &arr[i][j]);
        }
    }
}
```

3. Multidimensional Array in C:

In C language, we call Multidimensional array in simple words array of arrays, in which data is stored in tabular form (in the form of rows and columns).

Syntax: data_type array_name[size1][size2]....[sizeN];

Example: int three_dArray[2][3][4];

Initializing Three Dimensional Arrays In C:

```
int three_dArray[2][3][4] = {  
    {{3, 4, 5, 7}, {1, -3, 10, 11}, {3, 12, 53, 20}},  
    {{17, 4, 16, 3}, {56, 9, 31, 5}, {30, 1, 42, 9}}};
```

```
#include <stdio.h>  
int main()  
{ int x,y,z;  
  int arr[2][3][2] = {  
      { {1,-5},{2,-6}, {3,-7} },  
      { {4,-8},{5,-8}, {6,-10} }  
  };  
  for ( x=0;x<2;x++)  
  {  
      for ( y=0;y<3;y++)  
      {  
          for ( z=0;z<2;z++)  
          {  
              printf("%d  ", arr[x][y][z]);  
          }  
          printf("\n");  
      }  
      printf("\n");  
  }  
  return 0;  
}
```


String in C:

- A String is an array of characters that is used to manipulate text such as words and sentences. In simple words, “Sequence of characters which is terminated by null character (‘\0’) is called string.

Note : The null character (‘\0’) is not a printable symbol, it is like a space, due to which we do not need to know how many characters are in the string. We can easily tell where a string ends by looking at the null character (‘\0’).

Declaration of String in C: `char str_name[size];`

- Char: It is a character data type which indicates that we can store only character in the variable created with its help.
- str_name: The name of the variable that you are going to create will come in place of this str_name.
- size: It tells the size of the string, if 20 is written here then the size of char string variable will be 20 and it will be able to store only 20 characters. You can define the size of the string variable according to the size of the string.

- **To output the string, you can use the printf() function together with the format specifier %s.**

```
char greetings[] = "Hello World!";  
printf("%s", greetings);
```

Access Strings:

Since strings are actually arrays in C, you can access a string by referring to its index number inside square brackets [].

```
char greetings[] = "Hello World!";  
printf("%c", greetings[0]);
```

This example prints the first character (0) in greetings

Loop Through a String:

```
char carName[] = "Volvo";  
int i;  
  
for (i = 0; i < 5; ++i) {  
    printf("%c\n", carName[i]);  
}
```

String Functions:

- `#include <string.h>`
- String Length

`strlen()` function:

```
Ex: char alphabet[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
    printf("%d", strlen(alphabet));
```

- In the Strings, we used `sizeof` to get the size of a string/array.
- `sizeof` and `strlen` behaves differently, as `sizeof` also includes the `\0` character when counting.

```
Ex: char alphabet[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
printf("%d", strlen(alphabet));  
printf("%d", sizeof(alphabet));
```

- **Concatenate Strings:**

To concatenate (combine) two strings, we can use the `strcat()` function.

```
char str1[20] = "Hello ";  
char str2[] = "World!";
```

```
// Concatenate str2 to str1 (result is stored in str1)  
strcat(str1, str2);
```

```
// Print str1  
printf("%s", str1);
```

- **Copy Strings:**

To copy the value of one string to another, we can use the `strcpy()` function.

```
char str1[20] = "Hello World!";  
char str2[20];
```

```
// Copy str1 to str2  
strcpy(str2, str1);
```

```
// Print str2  
printf("%s", str2);
```

- **Compare Strings**

- To compare two strings, you can use the strcmp() function.
- It returns 0 if the two strings are equal, otherwise a value that is not 0.

```
char str1[] = "Hello";  
char str2[] = "Hello";  
char str3[] = "Hi";
```

```
// Compare str1 and str2, and print the result  
printf("%d\n", strcmp(str1, str2)); // Returns 0 (the strings are equal)
```

```
// Compare str1 and str3, and print the result  
printf("%d\n", strcmp(str1, str3)); // Returns -4 (the strings are not equal)
```

Escape Character	Result
\n	New Line
\t	Tab
\0	Null

Escape character	Result	Description
\'	'	Single quote
\"	"	Double quote
\\	\	Backslash

Type casting:

- Type casting is the process in which the compiler automatically converts one data type in a program to another one. Type conversion is another name for type casting.

Syntax: (type)value;

- **Without Type Casting:**

```
int f= 9/4;
```

```
printf("f : %d\n", f );//Output: 2
```

- **With Type Casting:**

```
1. float f=(float) 9/4;
```

```
2. printf("f : %f\n", f );//Output: 2.250000
```

- **To cast int value into the float.**

```
#include<stdio.h>
```

```
int main(){
```

```
float f= (float)9/4;
```

```
printf("f : %f\n", f );
```

```
return 0;
```

```
}
```

There are two types of conversion in C:

1. Implicit Conversion (automatically)
2. Explicit Conversion (manually)

- **Implicit Conversion**

Implicit conversion is done automatically by the compiler when you assign a value of one type to another.

// Automatic conversion: int to float

```
float myFloat = 9;
```

```
printf("%f", myFloat); // 9.000000
```

- **Explicit Conversion**

Explicit conversion is done manually by placing the type in parentheses () in front of the value.

// Manual conversion: int to float

```
float sum = (float) 5 / 2;
```

```
printf("%f", sum); // 2.500000
```

Formatted and unformatted Input/Output-

- **Formatted I/O Functions:** It is used to take various inputs from the user and display multiple outputs to the user. These types of I/O functions can help to display the output to the user in different formats using the format specifiers.

These I/O supports all data types like int, float, char etc. we can format these functions according to our needs.

- **Formatted I/O functions:** printf(), scanf().

Unformatted I/O functions: It is used only for character data type or character array/string and cannot be used for any other datatype. These functions are used to read single input from the user at the console and it allows to display the value at the console.

We cannot use format specifiers in these functions and hence, cannot format these functions according to our needs.

- **Unformatted I/O functions:** gets(), puts(), getch(), putch().

printf(): printf() function is used in a C program to display any value like float, integer, character, string, etc on the console screen. It is a pre-defined function that is already declared in the stdio.h(header file).

Syntax: To display any variable value.

printf(“Format Specifier”, var1, var2,, varn);

```
// C program to implement
// printf() function
#include <stdio.h>

// Driver code
int main()
{
    // Declaring an int type variable
    int a;

    // Assigning a value in a variable
    a = 20;

    // Printing the value of a variable
    printf("%d", a);

    return 0;
}
```

Syntax: To display any string or a message.

printf(“Enter the text which you want to display”);

```
// C program to implement
// printf() function
#include <stdio.h>

// Driver code
int main()
{
    // Displays the string written
    // inside the double quotes
    printf("This is a string");
    return 0;
}
```

- **scanf():** scanf() function is used in the C program for reading or taking any value from the keyboard by the user, these values can be of any data type like integer, float, character, string, and many more. This function is declared in stdio.h(header file), that's why it is also a pre-defined function.

In scanf() function we use (&) which is used to store the variable value on the memory location of that variable.

Syntax: scanf("Format Specifier", &var1, &var2,, &varn);

```
// C program to implement
// scanf() function
#include <stdio.h>

// Driver code
int main()
{
    int num1;

    // Printing a message on
    // the output screen
    printf("Enter a integer number: ");

    // Taking an integer value
    // from keyboard
    scanf("%d", &num1);

    // Displaying the entered value
    printf("You have entered %d", num1);

    return 0;
}
```

- **gets():** gets() function reads a group of characters or strings from the keyboard by the user and these characters get stored in a character array.

This function allows us to write space-separated texts or strings. This function is declared in stdio.h(header file).

Syntax: char str[length of string in number]; //Declare a char type variable of any length

gets(str);

```
#include<stdio.h>
```

```
void main ()
```

```
{
```

```
    char s[30];
```

```
    printf("Enter the string ");
```

```
    gets(s);
```

```
    printf("You entered %s",s);
```

```
}
```

- **puts():** The puts() function is very much similar to printf() function. The puts() function is used to print the string on the console which is previously read by using gets() function.

The puts() function returns an integer value representing the number of characters being printed on the console.

Syntax: int puts(char[])

```
// C program to implement
// the puts() function
#include <stdio.h>

// Driver code
int main()
{
    char name[50];
    printf("Enter your text: ");

    // Reads string from user
    gets(name);

    printf("Your text is: ");

    // Displays string
    puts(name);

    return 0;
}
```

- **getch():** getch() function reads a single character from the keyboard by the user but doesn't display that character on the console screen and immediately returned without pressing enter key.

This function is declared in conio.h(header file).

getch() is also used for hold the screen.

Syntax: getch(); or variable-name = getch();

```
// C program to implement
// getch() function
#include <conio.h>
#include <stdio.h>

// Driver code
int main()
{
    printf("Enter any character: ");

    // Reads a character but
    // not displays
    getch();

    return 0;
}
```

- **putch():** putch() function is used to display a single character which is given by the user and that character prints at the current cursor location.

This function is declared in conio.h(header file).

Syntax: putch(variable_name);

```
// C program to implement
// the putch() functions
#include <conio.h>
#include <stdio.h>

// Driver code
int main()
{
    char ch;
    printf("Enter any character:\n ");

    // Reads a character from the keyboard
    ch = getch();

    printf("\nEntered character is: ");

    // Displays that character on the console
    putch(ch);
    return 0;
}
```

THANK YOU